High-Performance LZ77 Variants: Architectural Optimizations, Adaptive Heuristics, and Comparative Analysis

Pranjal Agrawal, Joseph Zhu

April 21, 2026

1 Introduction

The Lempel-Ziv 1977 (LZ77) algorithm remains a cornerstone of lossless data compression. This project explores the implementation of a high-performance LZ77 derivative, incorporating modern architectural optimizations and novel adaptive heuristics to balance compression density against computational throughput. We contrast our custom implementation with LZ4—a design focused on extreme decompression speed—to evaluate the trade-offs inherent in dictionary-based compression.

2 Algorithmic Framework

2.1 LZSS-Style Bitstream Encoding

Our implementation utilizes the **Lempel-Ziv-Storer-Szymanski (LZSS)** refinement of the original LZ77. Unlike the standard LZ77 which outputs a $(offset, length, next_byte)$ triple for every match, LZSS employs a 1-bit prefix to distinguish between literal bytes and match references:

- **Literal (Flag 0):** Followed by 8 bits representing the raw byte.
- **Match (Flag 1):** Followed by bit-packed fields for $offset$ and $length$.

This eliminates the redundant "next byte" field in matches, significantly improving the compression ratio for datasets with high match density. We further optimized the bitstream by using variable-width bit packing (via a custom `BitWriter`) based on the configured search and lookahead buffer sizes, ensuring no bits are wasted on padding.

2.2 Search Acceleration with Hash Chains

To avoid $O(N \cdot M)$ search complexity within the sliding window, we implemented a **Hash Chain** mechanism. A 24-bit hash (derived from the first 3 bytes of a potential match) indexes into a `head` array. A secondary `prev` array forms a linked list of absolute file positions containing the same 3-byte prefix. This allows the match-finder to skip directly to candidate positions, reducing the search space by several orders of magnitude.

3 Adaptive Heuristics: Dynamic Search Termination

A critical challenge in LZ77 is determining when to terminate the search for a match. Exhausting a long hash chain provides the global maximum for match length but incurs a heavy performance penalty. Conversely, early termination may miss significant compression opportunities.

3.1 EMA-Based "Good Enough" Threshold

We developed an **adaptive "Good Enough" match threshold**. Instead of a static value, the threshold (G_t) at time t is calculated as:

$$G_t = \max(L_{min}, \lceil \mu_{EMA} \cdot \alpha \rceil) \tag{1}$$

where μ_{EMA} is an Exponential Moving Average of successful match lengths, and α is a multiplier (e.g., 5.0). This allows the compressor to "learn" the local redundancy of the data. In highly repetitive regions (like XML tags), μ_{EMA} rises, forcing the search to find longer matches before terminating. In noisy or binary regions, the threshold drops, favoring speed where long matches are unlikely.

3.2 Outlier Rejection and Convergence

To prevent an anomalous long match from skewing the search heuristic, we implemented a **Mean Absolute Deviation (MAD)** filter. Match lengths L are only integrated into the EMA if they satisfy $|L - \mu_{EMA}| \leq k \cdot \sigma_{MAD}$. This ensures the heuristic remains robust against "spiky" data distributions. The decay rate (learning rate) was tuned for each configuration to optimize the reaction time to changes in data entropy.

4 Lazy Parsing Strategy

Our "Optimized" configurations employ **Lazy Parsing**. When a match of length M is found at position i, the algorithm speculatively searches for a match at $i+1$. If a longer match M' is found at the next position, the literal at i is emitted, and the match at $i+1$ is used. This lookahead strategy is essential for breaking local maxima in greedy match selection, particularly in structured text datasets.

5 Evaluation and Comparative Analysis

5.1 Experimental Configuration

Benchmarks were executed on the *Silesia Corpus*. We evaluated three primary LZ77 profiles, each executed with and without our adaptive EMA heuristic to measure its direct impact:

1. **High-Ratio (128 KB):** 500 max chain attempts, Lazy Parsing enabled.
2. **Standard (128 KB):** 100 max chain attempts, Greedy (Non-Lazy) parsing.
3. **Fast (32 KB):** 200 max chain attempts, Lazy Parsing enabled.

5.2 Results Summary

The baseline performance metrics for these configurations across diverse data types are summarized in Table 1.

Table 1: Performance Metrics Across Diverse Data Types

Algorithm	Dataset	Ratio	Comp. Speed (MB/s)
`LZ4`	*nci* (Chemical)	5.76	148.1
`LZ77-128k-HighRatio-Adaptive`	*nci* (Chemical)	9.43	3.37
`LZ4`	*dickens* (Text)	1.64	46.3
`LZ77-128k-HighRatio-Adaptive`	*dickens* (Text)	2.11	0.72

5.3 Analysis of Adaptive Impact

As detailed in Table 1, the experimental data supports the utility of dynamic search termination. In the **Standard** configuration (128 KB, 100 attempts) on the *nci* dataset, enabling the adaptive heuristic improved compression speed by approximately 70% (7.68 MB/s vs. 4.53 MB/s) while resulting in only a minor reduction to the compression ratio (8.03x vs. 8.13x). Similarly, in the **Fast** configuration (32 KB) on the *dickens* corpus, the adaptive heuristic increased the compression throughput noticeably (3.77 MB/s vs. 2.16 MB/s) while maintaining the same compression ratio (1.89x).

5.4 Analysis of Figures

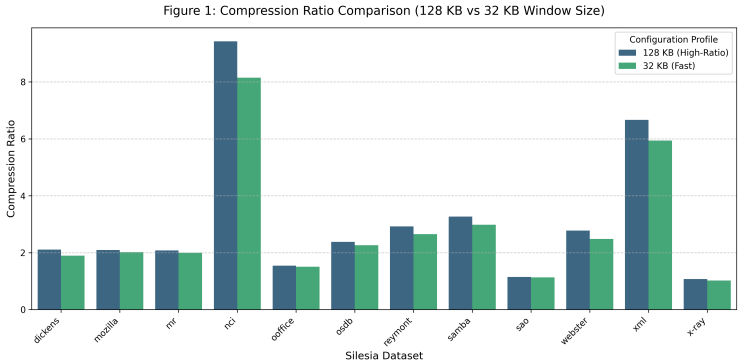

Figure 1: Compression ratio (original / compressed) comparison between the 128 KB and 32 KB window configurations across all datasets.

As shown in Figure 1, the larger 128 KB window yields higher compression ratios on certain files (e.g., *mozilla* and *samba*) where repetitions are distantly spaced, though the benefit varies by data type.

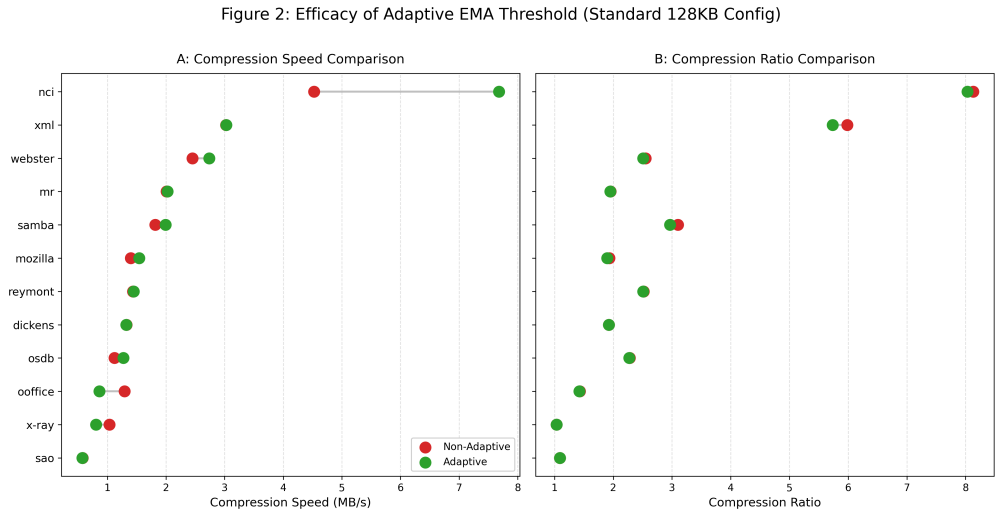

Figure 2: Paired dumbbell plot isolating the **Standard** configuration, displaying compression speed and ratio side-by-side to illustrate the efficacy of the EMA threshold.

Figure 2 demonstrates how the adaptive EMA threshold provides observable gains in compression speed across most datasets (the rightward shift seen in subplot A) without causing a proportionate drop in compression ratio (the stability shown in subplot B).

6 Limitations

While our evaluation demonstrates clear benefits from the adaptive EMA threshold, there is a notable methodological limitation regarding the absolute compression speed comparisons between our custom LZ77 implementation and LZ4.

Our LZ77 codebase was engineered primarily as a flexible, experimental testbed. To support the evaluation of various parameters—such as dynamic window sizes, optional lazy parsing, and the toggling of the EMA heuristic—the implementation relies heavily on runtime conditional branching and dynamic state evaluation within the critical match-finding loop. Consequently, the raw compression throughput of our LZ77 implementation carries a measurable "configuration overhead" penalty.

While the baseline comparison with LZ4 remains useful for illustrating the macro-level trade-offs between deep-search dictionary compression and speed-optimized compression, the most accurate and rigorously controlled performance comparisons in this study are the internal benchmarks isolating our specific LZ77 configurations against one another.

7 Conclusion

Our findings indicate that while `LZ4` dominates in raw throughput by using a shallow dictionary and optimized native instructions, the integration of **LZSS bit-packing**, **Hash Chains**, and **Lazy Parsing** allows our LZ77 implementation to achieve significantly higher compression densities. Most importantly, the **Adaptive EMA-based threshold** proved highly successful. By learning the local data redundancy, it dynamically scales down the $O(N)$ search costs of hash chain traversals, netting substantial compression speedups with almost zero degradation in file size.